# Série 4 — Conteneurs, architecture et commandes de base (Édition Windows)

**Université Hassan 1er — FST Settat**
**Département Informatique**
Année Universitaire : 2026–2027
**Module :** Cloud Computing, Virtualisation & DevOps
**Enseignant :** Pr. Abdelfattah Abassi

## Série 4 — Conteneurs, architecture et commandes de base (Windows)

Durée : 2 h (TD 40 min + TP 80 min)
Chapitre Conteneurisation — Docker Desktop, WSL 2 & PowerShell

### Objectifs de la série (Environnement Windows)

À l'issue de cette série, l'étudiant sera capable de :

- comprendre le fonctionnement de Docker sous Windows (moteur WSL 2 / Hyper-V) ;
- distinguer une machine virtuelle classique d'un conteneur Linux hébergé sous Windows ;
- utiliser la CLI Docker dans PowerShell et le sous-système Windows pour Linux (WSL) ;
- démarrer, lister, explorer, arrêter et supprimer des conteneurs sous Windows ;
- mesurer expérimentalement la légèreté des conteneurs comparés aux VM Hyper-V/VirtualBox.

## Partie TD (40 min)

### Exercice 1 — Docker sous Windows : WSL 2 ou VM Linux ? (10 min)

Indiquer si chaque affirmation est vraie ou fausse, puis corriger les affirmations fausses.

1. Sous Windows, Docker exécute directement des conteneurs Linux nativement sans sous-couche Linux.
2. Docker Desktop sous Windows s'appuie généralement sur un noyau Linux léger via WSL 2 (Windows Subsystem for Linux).
3. Les commandes `docker` exécutées dans PowerShell s'adressent au démon Docker s'exécutant dans WSL 2.
4. L'isolation d'un conteneur Linux sur Windows est assurée par le noyau Linux partagé dans l'instance WSL 2.
5. La consommation mémoire de Docker Desktop sous Windows est fixe et identique à celle d'une VM sous Hyper-V.
6. Un conteneur démarré sous Windows peut publier des ports accessibles via `localhost`.

**Question complémentaire :** Pourquoi est-il préférable d'utiliser le backend WSL 2 plutôt qu'une VM Hyper-V classique sous Windows pour exécuter Docker ?

### Exercice 2 — Architecture de Docker Desktop sous Windows (10 min)

1. Expliquer le rôle du client CLI (1) sous Windows et la manière dont il communique avec le démon (2).
2. Si Docker Desktop est fermé sous Windows, que se passe-t-il lorsque vous tapez `docker version` dans PowerShell ?
3. Quelle est la différence entre exécuter la commande `docker` dans un terminal PowerShell et

l'exécuter dans un terminal WSL 2 (Ubuntu) ?

### Exercice 3 — Image ou conteneur ? (10 min)

Un administrateur exécute trois fois la commande suivante sous PowerShell :

```
docker run -d nginx
```

1. Combien d'images `nginx` et combien de conteneurs existent-ils ensuite sur le système ?
2. Il supprime ensuite les trois conteneurs avec la commande `docker rm -f (docker ps -aq)` sous PowerShell. L'image existe-t-elle encore ?
3. Expliquer pourquoi les images Docker sont stockées au sein du système de fichiers virtuel VHDX de WSL 2 sous Windows.

### Exercice 4 — Redirection de ports sous Windows (10 min)

```
docker run -d --name monweb -p 8080:80 nginx
```

1. Expliquer chaque option de cette commande.
2. Dans votre navigateur Windows (Edge/Chrome), quelle URL saisissez-vous pour vérifier le bon fonctionnement ? Peut-on aussi utiliser `Invoke-WebRequest` sous PowerShell ?
3. Que se passe-t-il si le port 8080 est déjà utilisé par une autre application Windows (ex. un serveur IIS ou Jenkins) ? Comment corriger ?

## Partie TP (80 min)

### Attention Windows

S'assurer que **Docker Desktop** est démarré sous Windows (icône de la baleine présente dans la barre des tâches). Utiliser au choix **PowerShell** ou un terminal **WSL 2 (Ubuntu)** pour exécuter les commandes.

### Remarque Windows / PowerShell

Sous PowerShell, les guillemets et certaines redirections fonctionnent différemment de Bash. En cas de doute, la syntaxe standard Docker reste compatible sous Windows.

### Étape A — Vérifier l'installation sur Windows (5 min)

Ouvrir PowerShell et exécuter :

```
docker --version
docker version
docker info
docker run hello-world
```

Noter la version du client Windows, la version du moteur Linux (WSL 2), ainsi que les informations affichées par `docker info`.

### Étape B — Premier serveur web sous Windows (15 min)

```
docker run -d --name monweb -p 8080:80 nginx
docker ps
curl http://localhost:8080
docker port monweb
docker logs monweb
```

```
docker logs -f monweb
```

Ouvrir votre navigateur sous Windows et naviguer vers `http://localhost:8080`. Pour le suivi des logs (`docker logs -f`), ouvrir une seconde fenêtre PowerShell, réexécuter `curl http://localhost:8080` et observer les lignes de journal reçues. Interrompre avec `Ctrl+C`.

## Étape C — Exploration interactive du conteneur (15 min)

```
docker exec -it monweb bash
cat /etc/os-release
ls /usr/share/nginx/html
echo "<h1>Bonjour Docker depuis Windows</h1>" > /usr/share/nginx/html/index.html
exit
curl http://localhost:8080
docker inspect -f "{{.NetworkSettings.IPAddress}}" monweb
```

**Questions :** Quel système d'exploitation tourne dans le conteneur ? Quel est le noyau affiché par `uname -r` dans le conteneur ? Est-ce un noyau Windows ou Linux ?

## Étape D — Cycle de vie et persistance sous Windows (10 min)

```
docker stop monweb
docker ps
docker ps -a
docker start monweb
curl http://localhost:8080
docker rm -f monweb
docker run -d --name monweb -p 8080:80 nginx
curl http://localhost:8080
```

**Observer :** Que survient-il au contenu modifié (`index.html`) après un simple arrêt/redémarrage ? Et après une suppression suivie d'une réinstanciation ?

## Étape E — Application NoSQL (Redis) sous Windows (15 min)

1. Lancer un conteneur Redis en arrière-plan nommé `cache01` :
   ```
   docker run -d --name cache01 redis
   ```
2. Vérifier qu'il est en cours d'exécution avec `docker ps`.
3. Exécuter le client Redis directement dans le conteneur :
   ```
   docker exec -it cache01 redis-cli
   ```
4. Tester la commande `ping`, puis ajouter la clé `SET cours "DevOps Windows"` et la relire avec `GET cours`. Quitter avec `exit`.
5. Que se passe-t-il si vous tentez d'exécuter `redis-cli` directement dans PowerShell sans passer par `docker exec` ?

## Étape F — Performance et métriques comparatives (15 min)

Exécuter plusieurs conteneurs simultanés et observer l'allocation des ressources sous Windows :

```
docker run -d --name web1 -p 8081:80 nginx
docker run -d --name web2 -p 8082:80 nginx
docker ps
docker images
docker stats --no-stream
```

```
Measure-Command { docker run --rm alpine echo "bonjour" }
```

**Remarque Windows / PowerShell**

Sous PowerShell, `Measure-Command { ... }` permet de mesurer la durée d'exécution d'une commande (équivalent du `time` Linux).

Compléter le tableau comparatif :

| **Mesure** | **nginx** | **alpine** |
|---|---|---|
| Taille de l'image (`docker images`) | | |
| Mémoire RAM utilisée (`docker stats`) | | — |
| Temps de démarrage (PowerShell) | — | |
| Nombre d'images pour `web1` et `web2` | | |

Nettoyage final de votre environnement Windows :

```
docker rm -f web1 web2 monweb cache01
docker ps -a
```

**À rendre en fin de séance**

Une fiche de rendu synthétique (Format PDF ou Word) contenant :

1. Le tableau de l'étape F dûment complété avec vos mesures.
2. Les réponses aux questions des étapes B à E.
3. Une capture d'écran de votre navigateur Windows affichant la page personnalisée Nginx.
4. Explication synthétique du rôle de WSL 2 dans le fonctionnement de Docker Desktop sous Windows.